



总复习week16_1229

C++基本概念

- 是一种编译型语言
- 基本模块：函数
 - 总是从main函数开始运行

1. C++程序的基本组成单位是？

A. 语句 B. 函数 C. 表达式 D. 文件

- 数据类型

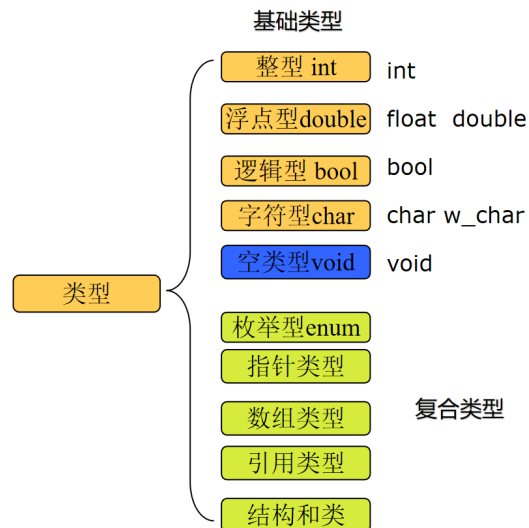
- 类型

- 整型int(4字节)
 - 修饰符long(6)/short(2)、signed/unsigned
 - 浮点型float
 - float(4)、double(8)、long double
 - 逻辑型bool(1)
 - 只有两个取值 true 1 和 false 0
 - 占1个字节byte (8个bit)
 - 字符型char(1)
 - char存储单个字符，实际存储的是编码值ASCII码
 - string字符串型以\0作为结束符，所以实际会多一个字符，多占1个字节的内存
 - 字符串和整型可以互相赋值，在一定范围内进行加减法（字符串型不可以）
 - “包裹字符型”，“”包裹字符串型

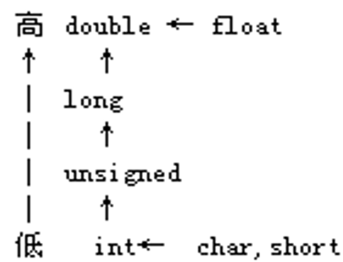
字符型	字符常量	用单引号括起来的字符。 转义字符 用“\”开头的特殊字符	如 'a'、'x'、'D'、'?'、'\$' 等 \n 换行 (new) \t 横向跳格 (table) \v 竖向跳格 (vertical) \b 退格 (backspace) \r 回车 (return) 指打印机 \f 走纸换页 (form feed) \\ 字符“\” \' 字符“'” \ddd 1到3位8进制数所代表的字符 \xhh 1到2位16进制数代表的字符
	字符变量	字符变量用来存放字符常量，只能存放一个字符	定义形式如下： char c1, c2 表示c1和c2为字符型变量， 各可以存放一个字符如c1='a'；
	字符串常量	是用一对双引号括起来的字符序列	如：“How do you do.” “CHINA”，“\$332.33” 注意用“”括起来的字符串中是 以'\0'作为字符串结束标志的 所以比所显示的多一个字符。

- 空类型void

- 枚举型enum
- 指针类型
- 数组类型
- 引用类型
- 结构和类



- 数据类型转换总是朝表达数据能力更强的方向发展



信息的表达和存储

- 最基本最小的单位：比特bit位
- 存储容量和数据处理的基本单位：字节byte（8个bits，256种不同状态0-255）
- 变量命名
 - _字母数字组成，不以数字开头
 - 不能与关键字（都是保留字故不能使用作为名称），库函数，类名，对象名重名
 - 不能超过31个字符

*大小写敏感

*define是 C/C++ 的预处理宏指令（格式：#define 宏名 替换值），并非语言内置关键字

C++ Keywords				
<i>Keywords common to the C and C++ programming languages</i>				
auto	break	case	char	const
continue	default	do	double	else
enum	extern	float	for	goto
if	int	long	register	return
short	signed	sizeof	static	struct
switch	typedef	union	unsigned	void
volatile	while			
<i>C++ only keywords</i>				
asm	bool	catch	class	const_cast
delete	dynamic_cast	explicit	false	friend
inline	mutable	namespace	new	operator
private	protected	public	reinterpret_cast	
static_cast	template	this	throw	true
try	typeid	typename	using	virtual
wchar_t				

- 变量所占内存大小
 - sizeof确定某数据类型的字节长度
- ASCII码表
 - 字符型和整型的关系

ASCII字符代码表

- 大小写转换: 'b'-'a'+ 'A'='B'

变量、常量、运算符

- 变量：可以改变的量
- 常量：不能改变的量
 - const：C++关键字，定义的是具有类型和作用域的常量对象，占用内存，有明确数据类型（const float pi=3.1415926;）
 - #define：预处理器指令（不是关键字），在编译前由预处理器进行文本替换，编译时就不存在改标识符了。是宏不是常量，不占用内存，没有数据类型，不能进行赋值（#define PI 3.1415926）
 - const（常变量使用const）和#define（符号变量）区别
 - const可以在局部范围使用
 - #define一经定义全局有效
 - const更加安全
- 运算符
 - 算数运算符 () >> + - - >> */% >>+-
 - 关系运算符 （区分=和==）
 - 逻辑运算符 ! >> && >> ||



2. 表达式 $3 \& 5$ 的值是 0001 (用十进制表示)。

先把十进制转化成二进制 (除以二取余数, 如果商不为0, 拿商继续除以二取余, 直至商为0, 倒序取余为二进制数), 从右往左逐位比较, 同为1才为1, 其他为0

4. 表达式 $5 > 3 > 1$ 的值是 0。

4.

答案: 0

解析: C++ 中关系运算符是左结合的:

1. 先计算 $5 > 3$, 结果为 true (C++ 中用 1 表示);
2. 再计算 $1 > 1$, 结果为 false (C++ 中用 0 表示)。

已知 `int a = 5, b = 2;`, 则表

达式 `++a * b--` 的值是?

- A. 10
- B. 12
- C. 15
- D. 18

答案: B

`++a` 是前置自增, `a` 先自增为 6, 然后参与运算。
`b--` 是后置自减, `b` 的当前值 2 先参与运算, 然后自减为 1。因此表达式变为 $6 * 2 = 12$

若有 `int x = 5;`,

则执行语句 `x += x -= x * x;`

后, `x` 的值是?

- A. 0
- B. -15
- C. -20
- D. -40

答案: D

赋值运算符 (包括复合赋值) 的结合性为从右向左。首先计算 `x * x` (即 $5 * 5 = 25$)。然后计算 `x -= 25` (即 $x = 5 - 25 = -20$)。此时 `x` 的值为 -20。最后计算 `x += -20` (即 $x = -20 + (-20) = -40$)

3. 表达式 $8 \parallel 0 \&\& 1$ 的值是 1。

3. 表达式(10 > 5) || (3 < 2) && (7 == 7)的值是?

A. 0 B. 1 C. true D. false

C++ 中逻辑表达式的返回值是 `int` 类型的 `1/0`，而非 `bool` 类型的 `true/false`，因此选 B。)

4. 若要将整数 255 以十六进制形式输出，应使用的格式控制符是 `%hex`。

过程化语言是三种形式：顺序、选择/分支、循环

结构化程序设计所规定的三种基本控制结构是顺序、选择和递归

错误	结构化程序设计的三种基本控制结构是 “顺序结构、选择结构 (if/switch)、循环结构 (for/while/do-while)”；递归是 “函数调用自身的编程技巧”，可通过循环结构实现，不属于基本控制结构，故说法错误。
----	--

条件语句

- if

```
if (条件表达式)
{
    语句块1;
}
else //else 总是与最近的未配对 if 匹配
{
    语句块2;
}
```

- switch (考点: break; default)

```
switch(表达式) //没有分号冒号
{
    case 常量表达式1:
        语句块1;
        break; 表示跳出来，没有break会继续做
    case 常量表达式2:
        语句块2;
        break;
    ...
    case 常量表达式n:
        语句块n;
        break;
    default:
```

```
    语句块n+1;  
}
```

```
switch(grade/10)  
{  
    case 10:  
    case 9:  
        cout<<"A";  
    case 8:  
        cout<<"B";  
    case 7:  
        cout<<"C";  
    case 6:  
        cout<<"D";  
    default:  
        cout<<"F";  
}
```

/*核心原因：switch 的 “case 穿透” 现象

这段代码的关键问题是所有 case 分支都没有写break语句，导致程序执行完匹配的 case 后，不会跳出 switch 结构，而是继续执行后续所有 case 和 default 的代码。

分步分析执行过程

计算grade/10：85 是整数，在 C++ 中整数除法会舍弃小数，所以 $85/10 = 8$ ；

switch 匹配case 8，执行cout<<"B"，输出B；

没有break，继续执行下一个case 7，输出C；

依然没有break，继续执行case 6，输出D；

还是没有break，执行default分支，输出F；

最终拼接的输出结果是：BCDF。*/

循环语句

- while （考点while()后面无;）

```
while (循环条件) {  
    // 循环体  
}
```

//适用于循环次数不确定，仅通过条件判断是否继续的场景

- do-while

```
do {  
    // 循环体  
} while (循环条件); // 注意末尾的分号
```

```
//无论初始条件如何，循环体至少执行一次
```

- for (for(括号里用;), 表达式3是循环体全部做完才执行)

```
for (初始化; 循环条件; 增量/减量) {  
    // 循环体 (要重复执行的代码)  
}
```

```
//适用于已知循环次数的场景
```

- 跳出循环
 - break: 彻底终止循环
 - continue: 跳过本次循环剩余部分，直接进入下一次迭代

函数

- 函数: 是指一段在一起的、可以完成一定功能的程序代码 (封装特定功能的可重复代码块, 以实现一个特定的功能)
- 包括用户自定义函数、库函数
- 作用: 减少重复, 将程序拆分, 是程序清晰可读
 - 1 声明



返回类型 函数名 (输入参数表);
参数表必须明确参数类型、个数、顺序

- 2 调用

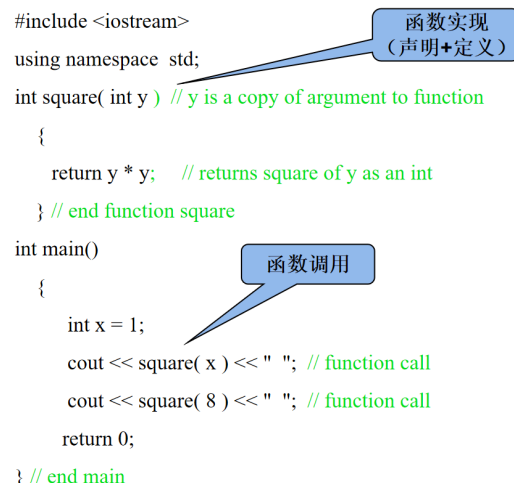
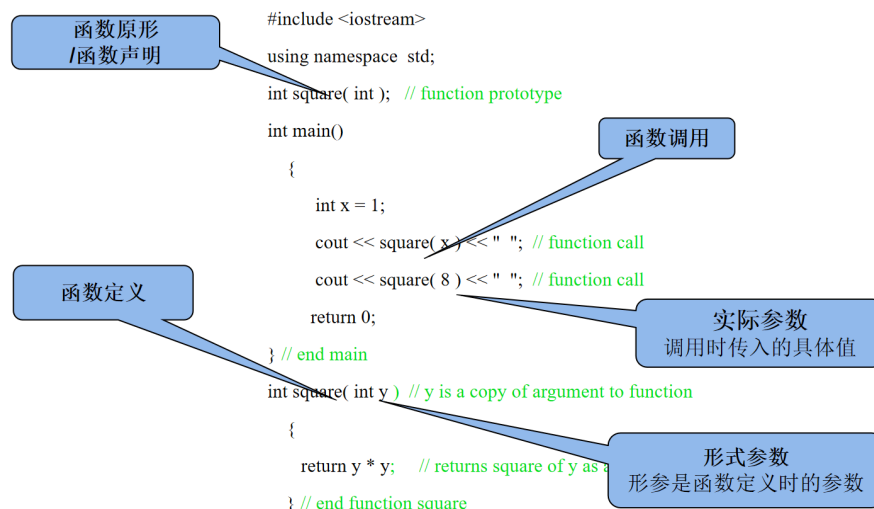


存放返回值的变量=函数名 (参数值);
参数值为实际计算时的值——实参

- 3 实现 (定义)



返回类型 函数名 (输入参数表)
{
 参数表必须明确每个参数在函数运算时的变量名称——形参
 //函数体
}



函数的返回值类型只能在函数定义时指定，不能在函数声明时指定。

错误	函数的返回值类型必须在声明和定义时保持一致，且声明时必须指定返回值类型（如 <code>int add(int a, int b);</code> ）。若声明时不指定返回值类型，编译器会默认为 <code>int</code> 类型，可能导致类型不匹配错误，故“不能在声明时指定”的说法错误。
----	--

函数声明时，必须提供函数的完整实现。

错误	函数声明的核心作用是“告知编译器函数的名称、返回值类型、参数列表”，无需提供完整实现；完整实现仅需在函数定义时编写。例如声明 <code>void print(int a);</code> ，定义时才写 <code>void print(int a) { cout << a; }</code> ，故说法错误。
----	---

变量的作用域

• 全局变量

- 在程序顶部定义，在程序的任何地方都可知
- 在所有函数外部定义的变量（包括 `main` 函数）
- 作用域：从定义位置到整个源文件结束（`extern` 声明可跨文件访问）即：所有函数内部、代码块内部，只要在全局变量定义之后，都能访问它

- 生命周期：程序启动时创建，结束时销毁
- 存储位置：全局数据区（而非栈区）
- 初始化：自动初始化为0，可以不初始化

• 局部变量

- 在函数内部定义，在该函数内部可知
- 在函数内部、代码块定义的变量
- 作用域：仅在定义它的函数或代码块内部有效，超出范围后无法访问（“出了大括号就失效”）
- 生命周期：从进入作用域时创建（如进入函数、进入代码块），到离开作用域时销毁（如函数执行结束、代码块执行结束）。
- 存储位置：栈区（函数调用时临时分配，结束后释放）。
- 初始化：不初始化，值为随机



- ①main中定义的变量，只在main中有效。
- ②不同函数中可以定义同名的变量，它们互不干扰。
- ③函数形参是函数的局部变量。



- ④同一个cpp文件中，全局与局部变量同名时，外部变量会被屏蔽，即局部优先。
- ⑤函数内部的复合语句中也可定义局部变量，有效范围在该复合语句中。

下列关于变量作用域的说法，正确的是（ ）

C

- A. 函数内部定义的变量在整个程序中有效
- B. 复合语句（{ } 内）定义的变量在语句外仍可访问
- C. 全局变量和局部变量同名时，局部变量优先
- D. 函数的形参是全局变量

静态局部变量

- 定义
 - static 类型 变量名；
 - static int a,b;（static a;是错误的）
- 特点
 - 静态局部变量属于静态存储变量，可以一直保留值
 - 静态局部变量只赋初值一次，每次调用都保留上次调用结束时的值

- 静态局部变量函数调用后仍然存在，但其他函数并不能引用！（static修饰的函数无法被其他源文件通过extern关键字引用）
- 不赋值，数值型自动为0，字符型为空字符
- 静态存储区的变量（static 修饰）在程序启动时分配内存，程序结束时释放
- static成员变量属于类本身，而非类的某个对象，需在类外单独初始化
- 局部变量声明为static，使该变量在函数调用结束时不释放，整个程序执行时都存在，使其存储期为程序的全过程。
- 全局变量用static，则该变量的作用域只限于本文件模块。
- **✗**普通全局变量加static后，存储位置从堆区变为静态存储区：普通全局变量本身就存于静态存储区，static仅修改其“链接属性”（从外部链接变为内部链接，限制作用域为当前源文件），不改变存储位置
- 应用
 - 需要保留函数上一次调用结束时的值

所有局部变量在使用前必须进行初始化。

错误	C++ 仅要求内置类型的局部变量若使用其值，必须初始化（否则为随机值，引发未定义行为）；若局部变量仅定义不使用（如int a;但后续无a的操作），编译器允许不初始化。且static局部变量会被编译器自动初始化为 0，并非“所有局部变量使用前必须手动初始化”，故说法错误。
----	---

关于 static 和 extern 存储类别，下列说法正确的是？

- A. static 全局变量的作用域是整个程序
- B. extern 用于声明一个在其他文件中定义的变量
- C. static 局部变量在每次函数调用时都会重新初始化
- D. extern 变量在声明时必须同时定义

正确答案：B

各选项分析：

- **选项 A：**错误。static 全局变量的作用域被限制在**定义它的文件内**（即“文件作用域”），无法被其他文件访问，并非整个程序。
- **选项 B：**正确。extern 的核心作用是**声明外部变量 / 函数**，即告诉编译器“该变量 / 函数在其他文件中定义”，用于跨文件共享。
- **选项 C：**错误。static 局部变量的生命周期是整个程序运行期，**仅在第一次函数调用时初始化一次**，后续调用不会重新初始化（保留上次的值）。
- **选项 D：**错误。extern 是“声明”而非“定义”，声明时不需要分配内存（如 extern int a; 是声明，int a = 10; 是定义），二者可分离。

9) 在函数内部定义变量时使用 static 修饰的变量，意味着函数调用结束后会被**保留**（销毁/保留）。

• 判断题

- 不同函数中不可以定义同名的变量，它们会互相干扰。✗
- 函数形参是函数的局部变量。✓
- 所有变量都必须先初始化、后使用。✗
- 静态局部变量在函数调用后仍然存在，其它函数能够引用。✗
- 函数声明时，必须指定函数的返回值类型、函数名和所有参数的具体实现。✗
- 在 C++ 中，函数参数传递方式为值传递时，实参的值会被复制一份给形参，函数内部对形参的修改不会影响实参本身。✓

假定函数定义为：

```
void fun(char ch, float x){ ... }
```

那么，正确的调用语句是

- A. fun('65', 2.8);
- B. t=fun('D', 16.5);
- C. fun("abc", 3.0);
- D. fun(32, 32);

D

下列函数声明完全正确的是：（ ）

- A. float fun(int x,int y)
- B. float fun(int x, y);
- C. float fun(int x,int y);
- D. float fun(int ,int)

C

若有以下函数调用语句:f(m+n,x+y,f(m+n,z(x,y)));在此函数调用语句中实参的个数是

- A. 6
- B. 5
- C. 4
- D. 3

D



函数调用中，实参是由“最外层逗号”分隔的独立表达式

递归函数

- 函数直接或间接调用自身叫函数的递归调用
- 解决具有自相似结构问题的工具，比如阶乘、斐波那契数列、树的遍历、分治算法

- 基准条件
 - 递归必须有一个或多个终止条件，防止无限调用
 - 一旦满足基准条件，函数不再递归，直接返回结果。
- 递归关系

函数调用自身，但每次调用都应使问题规模缩小，逐步逼近基准条件。
- 适合于
 - 问题具有自相似结构
 - 有明确的终止条件（否则栈溢出，递归过深也会栈溢出）
 - 每次递归逼近基准
- 递归与迭代（循环）有什么异同？
 - 重复
 - 迭代：清晰的循环
 - 递归：重复函数的调用
 - 终止条件
 - 迭代：循环条件不满足
 - 递归：回到最基本的情况
 - 都是有限循环
 - 迭代运算性能更好（效率高），递归更体现软件工程规范（费内存）
 - 递归函数因频繁的函数调用、栈帧创建 / 销毁，效率低
- （ ）是正确的递归函数。
 - A. `int Fun(int n) {if(n<1) return 1; else return n*Fun(n+1);}`
 - B. `int Fun(int n) {if(abs(n)<1) return 1; else return n*Fun(n/2);}`
 - C. `int Fun(int n) {if(n>1) return 1; else return n*Fun(n*2);}`
 - D. `int Fun(int n) {if(n>1) return 1; else return n*Fun(n-1);}`

答案：B

内联函数

- 通过在编译时将函数体直接插入到调用处来减少开销的优化机制（不用调用）。主要用于替代宏函数（`#define`），同时保持类型安全和调试便利性
- 格式：`inline` 返回类型 函数名（参数表）
- 优点：减少函数调用次数（快），适用小的频繁调用的函数
 - 但增加了目标程序的长度，所以才适用于小而频繁的函数
- 适用于：小的、频繁调用的函数

- 举例：inline double cube(const double s)

```
{ return s * s * s; }
```

使用 inline 关键字声明的函数一定会被编译器内联展开。

答案：对

默认参数的函数

- 在声明或定义函数时，为函数的一个或多个参数指定默认值的函数。当调用该函数时，如果没有为这些参数提供实参，编译器会自动使用预先指定的默认值；如果提供了实参，则使用传入的实参覆盖默认值。
- 指定默认值要从参数表右端开始
- 默认值可以是数值，也可以是表达式
- 只能在声明的时候写默认值，实现的时候不能再写
- 如果定义在调用之前，则定义时就要给出默认值；如果定义在调用之后，则调用前必须有函数声明并给出默认值，也就是说调用之前必须将默认值通知编译系统，也就是说默认值是在编译时确定的。
- 调用时可以为默认参数提供实参，会覆盖默认值

关于带有默认参数的函数，下列说法正确的是？

- A. 默认参数可在定义和声明中同时指定
- B. 默认参数必须放在参数列表最前面
- C. 默认参数的值在编译时确定
- D. 调用时不能为默认参数提供实参

C 默认参数的值在编译期确定，属于编译期常量，C 选项正确。A 选项“不可同时在声明和定义中指定”（否则编译错误）；B 选项“默认参数必须从右到左连续指定”（不能跳过中间参数）；D 选项“调用时可显式为默认参数传参”（覆盖默认值），均为错误说法。

重载函数

- 允许使用一个函数名说明几个函数，一个函数在其名被重用，被称为重载函数。
- 重载函数至少存在参数的个数、类型、顺序不同
- 返回值不同不能决定是否重载函数
- const 修饰符不可以作为区分依据

下面哪种情况可以构成合法的函数重载？

- A. 不同的返回类型但参数列表相同
- B. 相同的函数名和相同的参数列表，但在不同的命名空间中
- C. 相同的函数名，但一个有const修饰符而另一个没有
- D. 相同的函数名，参数数量相同但参数类型的顺序不同

答案：D

函数重载时，参数的 const 修饰可以作为区分依据（如 void f(int) 和 void f(const int)）。

答案：错

函数模板

- 函数模板是一种泛型编程机制，它允许编写与类型无关的通用函数，编译器会根据调用时的实际参数类型自动生成对应的具体函数版本（实例化）

```
template<typename/class T(数据类型参数标识符)>
<返回类型><函数名>(参数表)
{
    函数体
}
```

/*template 是关键字，表示这是一个模板。

typename（也可以用 class）用于声明一个模板参数（如 T）。

T 是一个占位符，代表任意类型，在函数体内可以像普通类型一样使用。*/

- 函数模板的核心思想是泛型编程：编写与类型无关的代码，编译器会在需要时根据实际使用的类型自动生成对应的函数版本



- ①函数模板不是函数，它是一个用于生成函数的“蓝图”。
- ②模板代码通常放在头文件中（因为编译器需要看到完整定义才能实例化）。
- ③如果函数体内对类型 T 有操作（如 +、>），则要求实际类型支持这些操作，否则编译报错。
- ④函数模板支持特化和重载，但需谨慎使用以避免二义性。
- ⑤如果存在同名的普通函数和函数模板，C++ 会优先选择普通函数（精确匹配）。
- ⑥函数模板是重载的一种补充机制。

4. 函数模板 `template<typename T> T max (T a, T b){ return a > b ? a : b; }` 的主要优点是？

- A. 提高了程序的运行效率
- B. 可以用于比较任意类型的两个数据
- C. 减少了源代码的行数
- D. 避免了函数重载的需要

正确答案：C

各选项分析：

- 选项 A：错误。函数模板在编译时会“实例化”为具体类型的函数（如 `int max(int, int)`、`double max(double, double)`），其运行效率与直接编写对应类型的重载函数完全一致，并未提高运行效率。
- 选项 B：错误。该模板依赖 `>` 运算符实现比较，仅支持**已重载 `>` 运算符的类型**（如内置类型 `int/double`、自定义类型需手动重载 `>`），无法用于“任意类型”（如未重载 `>` 的自定义类）。
- 选项 C：正确。若需支持 `int/double/float` 等多类型的 `max` 函数，原本需编写多个重载函数；使用函数模板仅需 1 段代码即可覆盖多类型，显著减少了源代码行数。
- 选项 D：错误。函数模板仅减少了“通用类型”的重载需求，但对于特殊类型（如需自定义比较逻辑的 `char*`、无 `>` 运算符的自定义类），仍需通过**模板特化**或函数重载实现，无法完全“避免函数重载的需要”。

11. 函数模板的主要作用是实现？

- A. 运行时多态
- B. 编译时多态**
- C. 提高代码执行速度
- D. 减少内存占用

数组

- 一种内置的数据类型，用于连续存储多个相同数据类型的元素，这些元素共享一个统一的变量名，通过下标索引进行区分

！考点：越界

- 同类型性：所有元素必须是相同数据类型（如 `int`、`double`、`char` 等），不能混合存储
- 固定长度：数组定义时必须指定长度（常量），一旦创建，长度不能修改
- 下标访问：通过“数组名 [下标]”访问单个元素，下标从 0 开始（而非 1）

数组在内存中的排列

- 整体连续：整个数组占用一块连续的内存空间，**数组名代表这块空间的起始地址/基地址（即第一个元素的地址）**
- 元素有序：数组元素按照“下标从 0 到 n-1”的顺序依次排列，相邻元素之间紧密衔接（无空隙），每个元素占用的字节数由数据类型决定



地址 of `arr[i]` = 基地址 + `i * sizeof(元素类型)` // `int` 4 字节，`double` 8 字节，`char` 1 字节
比如：`int` 数组，首地址 `0x100`，计算 `arr[2]`
 $0x100 + 2 * 4 = 0x108$

十六进制加减法：

0-15 对应 0-9-F

先将偏移量转为十六进制，再将个位相加——满十六进一——高位相加，重复操作

字符、字符数组和字符串

- 字符

- 表示单个字符，用单引号包裹
- 实际是整数类型，通常1字节，存储ASCII值
- 字符数组
 - 表示多个字符，单个字符的元素用单引号包裹，而整体初始化字符串时用双引号包裹
 - `char[]`或`char[N]`，存储在连续内存块
 - 单个字符要记得加`\0`，否则会一直读取内存，字符串赋值编译器会自动加`\0`



```
char dog[5]={ 'd','o','g','.'}
char cat[5]={ 'c','a','t','\0'}
```



dog没有以 `'\0'` 结尾！程序会继续读取 dog 之后的内存（可能是栈上的其他变量、垃圾数据等），直到偶然碰到一个字节值为 0 的位置才停止。

6. 若有定义 `char str[] = "hello";`，则 `sizeof(str)` 和 `strlen(str)` 的值分别是？

A. 5, 5 B. 6, 5 C. 5, 6 D. 6, 6

正确答案：B (6, 5)

分析：

1. `sizeof(str)` 的值：

`sizeof` 用于计算数组的**总字节数**。字符串常量 "hello" 会自动在末尾添加 `'\0'`（字符串结束标志），因此 `char str[] = "hello";` 实际存储的是 `'h','e','l','l','o','\0'`，共 6 个字符，故 `sizeof(str) = 6`。

2. `strlen(str)` 的值：

`strlen` 是字符串处理函数，用于计算字符串的**实际有效长度**（不包含末尾的 `'\0'`）。"hello" 的有效字符为 5 个，故 `strlen(str) = 5`。

- C++字符串
 - 以`\0`结尾的字符数组（`#include <cstring>`）
 - 动态长度的文本（`#include <string>`）
 - 常用函数
 - `strlen`（字符串）

字符串长度，**不包括结束符**
但是包括空格字符
 - `strcpy_s`（字符串1，长度N，字符串2）

字符串复制，字符串2复制给字符串1，复制长度为长度N。成功返回0，不成功返回一各错误代码。

- `strcat_s` (字符串1, 长度N, 字符串2)
字符串连接, 字符串2连接在字符串1后面, 复制长度为长度N。成功返回0, 不成功返回一各错误代码
- `cin.getline`
能够接受一行的输入, 不会把空格作为字符串结束标志。但输入的回车符, 会被认为是一个字符串。
- `cin.get`
不会把空格、回车符作为字符串结束标志。

• **iostream中读取含空格的整行输入, 应使用:**

- A. `cin >> str;`
- B. `gets(str);`
- C. `cin.getline(str, 100);`
- D. `getline(cin, str);`

C

向函数传递数组

```
#include <iostream>
using namespace std;
int sum(int array[], int len);
int main()
{
    static int ia[5] = {2, 3, 6, 8, 10};
    int sumOfArray = sum(ia, 5); //传递整个数组到一个函数,只需传递名字, 不要加括号
    cout << "sum of array: " << sumOfArray << endl;
    return 0;
}
int sum(int array[], int len)
{
    int iSum = 0;
    for (int i = 0; i < len; i++)
        iSum += array[i];
    return iSum;
}
```



函数内部无法通过 `sizeof(array)` 获取数组大小 (会得到指针大小, 如 8)
因此必须**显式传递长度 len**



数组作为函数参数

- ① 数组传递的本质：退化为指针，无论函数形参写 `int arr[]`、`int arr[10]` 还是 `int arr[100]`，本质都是 `int* arr`（接收数组首元素的地址！也不是首元素的值！），不会传递整个数组
- ② 数组传递是「地址传递」，函数形参 `arr` 接收的是原数组首元素的地址，函数内部 `arr[i]` 等价于「通过首地址找到原数组的第 *i* 个元素」（共享内存）
- ③ 函数形参中数组可以不指定长度，直接写 `int arr[]`（本质还是 `int* arr`）

指针

- 指针是一个变量，其值为另一个变量的内存地址
- 声明：数据类型 * 指针变量名 用*来定义指针变量

每一种数据类型都有相应的指针类型

```
char *s;
int *a,*b; //定义了两个指针
int *a,b; //定义了一个指针，一个整型变量
const int *y;
```

- 指针变量赋值，也可以通过指针修改变量值

```
int a = 10;
int *p = &a;

/* &取地址，表示变量的地址
   *解引用，用于访问指针所指向的值，表示变量的内容 */

*ptr = 100;
```



指针在使用前，需要进行初始化，否则会是野指针（语法上可以不初始化，所以**引用必须初始化**，指针不一定）

可以初始化指针为空，即空指针 `nullptr`

不能对空指针和未初始化指针解引用

可以对空指针使用 `delete`，不会导致程序崩溃

若有说明： `int a=2, *p=&a, *q=p;`，则以下非法的赋值语句是（ ）。

A、 `p=q;` B、 `*p=*q;` C、 `a=*q;` D、 `q=a;`

q是int*类型（指针），a是int类型（普通变量），指针不能直接赋值普通数值（需取地址q=&a），D选项非法。

- D
- A 选项 “p=q”（同类型指针赋值，合法）；
 - B 选项 “*p=*q”（指针解引用赋值，合法）；
 - C 选项 “a=*q”（将指针指向的值赋给变量，合法）。

11. 已知 `int a = 5; int *p = &a; int &ref = a;`，下列哪个表达式是非法的？

- A. `*p = 10;`
- B. `ref = 10;`
- C. `p = &ref;`
- D. `ref = p;`

指针与数组

- 数组名本质上是一个指向首元素的常量指针

```
int arr[3]={10,20,30};
int *p = arr;    //等价与p=&arr[0]
cout<<*(p+1)<<endl; //输出20
cout<<p[2]<<endl;  //指针可以像数组一样使用下标，此处输出为30
```

//在程序执行期间，数组名表示的起始地址是常量，不可以被修改，所以arr不可以进行自增自减+
+--，但是可以做加减法，但p可以✓p++ ✗arr++

12. 若有定义 `int a[10], *p = a;`，则以下哪个表达式不能正确表示 `a[3]`？

- A. `*(a+3)`
- B. `*(p+3)`
- C. `p[3]`
- D. `*p+3`

14

若有定义 `int a[] = {2, 3, 4, 5, 6}; int *p = a + 3;`，则表达式 `*(p--) + *++p` 的值是多少？
A. 4 B. 5 C. 6 D. 结果未定义

正确答案：D（结果未定义）

解析：

C++ 标准中，在同一个表达式内对同一个变量进行多次“修改操作”（如 `p--` 和 `++p`），且操作之间没有“序列点”（如逗号、分号）时，表达式的执行顺序是未定义的。

具体到本题：

- 表达式 `*(p--) + *++p` 中，`p` 被同时进行了 `--` 和 `++` 修改；
- 编译器可以先执行 `p--`，也可以先执行 `++p`，不同编译器、不同优化级别会产生不同结果；
- 这种行为属于“未定义行为”，因此结果无法确定。

表达式	序列点	执行顺序	结果
<code>*(p--) + *++p</code>	无	随机（未定义）	不确定
<code>a=*p--; b=*++p; a+b</code>	有 (;)	先 a 后 b	固定为 40
<code>(a=*p--), (b=*++p), a+b</code>	有 (,)	先 a 后 b	固定为 40



后置自增自减优先级高于解引用*

- `*p--` - 先自减移动指针，再解引用
- `(*p)--` - 先解引用，再自减移动指针

前置自增自减优先级等于解引用*，按照从右往左顺序做

- `*++p` 先移动指针，再解引用
- `++*p` 先解引用，再移动指针

那其实就是从右往左看

const和指针的组合

常量指针（可以不初始化）	<code>const int* p</code>	const在*左边	*p（p指向的内容）是常量，不能通过p修改内容，也就是说，内容不可以改，指针可以改	<code>ptr = &b; // ✓</code> 可以让 ptr 指向 b <code>*ptr = 30; // ✗</code> 错误！不能通过 ptr 修改所指的 值
指针常量（必须初始化ptr）	<code>int* const p=&a</code>	const在*右边	p是常量，不能修改，即指针不能修改，但指向的内容可以修改	<code>*ptr = 30; // ✓</code> 可以通过 ptr 修改 a 的值（a 变成 30） <code>ptr = &b; // ✗</code> 错误！ptr 是常量，不能再指向 b
指向常量的常量指针	<code>const int* const p</code>	阿巴阿巴	内容和指针都不可改，p是常量，指向的内容也是常量	

对于 `const float* p = &f;`，以下哪几项合法？

- A. `p = nullptr;`
- B. `*p = 1.0f;`
- C. `delete p;`
- D. `float val = *p;`

A和D

哪两个声明完全等价？

- A. `const char* p;`
- B. `char const* p;`
- C. `char* const p;`
- D. `const char* const p;`

A和B

以下声明中，指针本身是常量的是？（可多选）

- A. `int const* p;`
- B. `int* const p = &x;`
- C. `const int* const p = &x;`
- D. `int* p;`

B和C

20. 关于 C++ 中的 `const` 限定符与引用、指针的结合，下列说法正确的是？

- A. `const int& r = x;` 中，`r` 是一个对常量的引用，因此 `x` 的值在此之后不能通过任何方式被修改。
- B. `int* const p = &x;` 声明了一个指向整型常量的常量指针。
- C. 一个非 `const` 对象的地址可以赋值给 `const int*` 类型的指针，这是一种安全的隐式转换，用于保护数据不被意外修改。
- D. `const int* const p = &x;` 表示 `p` 是一个常量指针，它所指向的内存地址不能改变，但它所指向的内容可以通过其他非常量途径修改。

正确答案：C

各选项分析：

- 选项 A：错误。`const int& r = x` 表示“不能通过引用 `r` 修改 `x`”，但 `x` 本身若不是 `const` 对象，仍可通过 `x` 自身（如 `x = 10`）或其他非 `const` 方式修改，并非“不能通过任何方式修改”。
- 选项 B：错误。`int* const p = &x` 的含义是“指针 `p` 本身是常量（不能改变指向），但指向的内容是普通 `int` 类型”；而“指向整型常量的指针”应写为 `const int* p`（指向的内容是 `const`，指针本身可修改）。该选项混淆了 `const` 的位置。
- 选项 C：正确。`const int*` 是“指向 `const int` 的指针”，表示“不能通过该指针修改指向的内容”。将非 `const` 对象的地址赋值给它是安全的隐式转换——既不会破坏数据，还能限制通过该指针修改数据，实现“保护数据不被意外修改”的目的。
- 选项 D：错误。`const int* const p = &x` 的含义是“指针 `p` 本身是常量（不能改变指向），且指向的内容也是 `const`（不能通过 `p` 修改）”。若 `x` 本身是 `const` 对象，则“所指向的内容无法通过任何方式修改”；即使 `x` 是非 `const` 对象，该选项的描述也忽略了“`const int*` 限制了通过指针修改内容”的核心语义，表述不准确。

动态内存分配

- 指针常用于管理堆内存

```
//new申请分配内存 内存耗尽，返回值为null
int *p;
p=new int;
p=new int[20];
```

```
//delete释放内存 当分配的内存不再使用时，必须释放，否则会导致内存泄漏
delete p;
delete [ ]p;
```

```
//编写一段代码，要求用户输入一个正整数，然后创建一个动态的int数组，其中包含的元素数目等于用户的输入值。
#include <iostream>
using namespace std;
int main()
{
    unsigned int size; //声明一个无符号整型变量
```

```

cout << "Enter Positive integer to express array size\n";
cin >> size;
int* dyn = new int[size]; //声明一个动态数组，分配size个内存大小

for(int i=0;i<size;i++) // 循环输入数组元素
cin >> *(dyn+i);      // *(dyn+i)相当于dyn[i]

for (int i = 0;i < size;i++) //循环访问数组内元素的值
cout << i << " " << *(dyn + i) << " " << dyn[i] << endl; //索引 指针访问的值 数组下标访问的值

delete[]dyn;

return 1;
}

```

18. 关于 C++ 中 new 和 delete 的使用规则，下列说法错误的是？

- A. 使用 new[] 动态分配的数组，必须用 delete[] 释放，不能用 delete
- B. 对 NULL 指针使用 delete 是安全的，不会导致程序崩溃
- C. 若用 new int(10) 分配内存，释放时需用 delete[] 匹配**
- D. 动态分配的内存若未手动释放，可能导致内存泄漏

字符指针和字符数组

- 字符数组：char arr[]="Hello";
 - 编译器自动计算长度（包括结尾的 \0）
 - 等价于 char arr[6] = {'H','e','l','l','o','\0'}
 - 在**栈**上分配内存（如果是局部变量）
 - 内容可以修改
- 字符指针：char*ptr = "Hello";
 - "Hello" 是一个字符串常量
 - 存储在**只读内存区**（通常是 .rodata 段）
 - ptr 是一个指针，指向该只读区域
 - 不能修改其所指向的内容（否则行为未定义，通常程序崩溃）应该写成：const char* p = "Hello"; 但依旧不能修改值，可以用复制的方法

```

char* ps;
ps = new char[16];

```

```
strcpy_s(ps, 16, "character point");
```

指针与二维数组

- 二维数组：一维数组的数组

`int arr[3][4];` //3行4列的二维数组，连续存储12个int，等价于一个包含3个元素的数组，每个元素又是一个包含4个int的数组



C++ 中定义二维数组时，必须指定「列的大小」，行的大小可以省略（仅在初始化时）。二维数组可以理解为“数组的数组”（比如 `int arr[行][列]`，每一行都是一个长度为「列」的一维数组），编译器需要知道“每一行有多少个元素”才能计算内存布局，因此列的大小是必须的。

若二维数组a 有m 列，则计算任一元素`a[i][j]`在数组中位置的公式为：（ ）。

- A. $i*m+j$ B. $j*m+i$ C. $i*m+j-1$ D. $i*m+j+1$

A 二维数组“行优先”存储，`a[i][j]`表示第i行第j列元素，前面有i行（每行m个元素），共 $i*m$ 个元素，加上第i行的j个元素，总位置为 $i*m + j$ ，A选项正确。【位置也是从0开始！】
B、C、D选项均错误计算存储位置。

- 数组指针

`int (*p)[4] = arr;` //指向含4个int的数组



`arr`作为数组名，被退化为一个指向首元素的首地址，而`arr`的首地址是`arr[0]`，`arr[0]`中又是一个包含4个int的数组，即`int[4]`。



`int *p[4];` 表示一个数组，装有4个整数指针（指针数组）：从右往左读，`p[4]`表示p是一个装有4个元素的数组；`*p[4]`表示四个元素是指针；`int *p[4]`表示指针指向的类型是int，即`int*`类型的指针。

`int (*p)[4];` 表示一个指针，指向装有4个int的数组（数组指针）：`(*p)`表示p是指针；`(*p)[4]`表示指针指向装有4个int元素的数组；

ps：数组下标[]优先级高于声明指针*

`int **p;` 指向int指针的指针❌不能直接指向静态二维数组。静态二维数组`int arr[3][4]`在内存中的布局是`[int int ... int int]`而`int **p`在期望指向的结构是`[int* int* ... int* int*]`。并且`arr`的类型是`int[3][4]`数组指针，`p`的类型是`int**`指向指针的指针

通过指针访问数组

`int (*p)[4]=arr;`

- `p[i][j]=10;` //等价于`arr[i][j]`
- `*(p[i]+j)=10;` //`p[i]`等价于`*(p+i)`，得到第i行（第i个一维数组）的首地址；+j在这个一维数组内，向后偏移j个int的位置；*解引用，访问该位置的值。

- `*(*(p+i)+j)=10;` //p+i使指针p跳过i个长度为4的int数组，指向第i行；`*(p+i)`对结果解引用，得到第i行的首地址；`*(p+i)+j`在第i行内，偏移j个int；`*(*(p+i)+j)`解引用得到该值

扁平化指针

初始化：`int *p = &arr[0][0];` 或 `int *p = arr[0];` //取得第一个元素的地址

访问元素：`p[i * 4 + j];` //手动计算偏移

声明	含义	能否指向 <code>arr[3][4]</code> ?
<code>int *p</code>	指向单个 int	✅ (可指向 <code>&arr[0][0]</code>)
<code>int (*p)[4]</code>	指向含 4 个 int 的数组	✅ (可指向 <code>arr</code> 或 <code>arr[0]</code>)
<code>int **p</code>	指向 int 指针的指针	❌ (不能直接指向静态二维数组)

函数指针 (不考)

引用

- 提供了一种为已有变量创建**别名**的方式。引用不是新定义一个变量，而是给已存在的变量起另一个名字，对引用的操作就是对原变量的操作。
- 引用本质上是一个自动解引用的常量指针

```
int sint = 5;
int &rint = sint;
//目标类型&应用名=变量
```



引用在声明时**必须被初始化**

对引用的改动，就是对目标的改动

引用只是一个声明，不是一个定义

特性	说明
必须初始化	引用声明时必须绑定到一个已存在的对象，不能“悬空”。 <code>int &r;</code> // ❌ 错误!
不可重新绑定	一旦绑定，不能再指向其他变量。 <code>int b = 30; ref = b;</code> // ❌ 不是重新绑定，而是把 b 的值赋给 a!
不占额外内存	引用本质上是编译器实现的“别名”，通常不占用额外存储空间（类似宏替换，但类型安全）。
类型必须匹配	<code>double d = 1.5; int &r = d;</code> // ❌ 错误! 类型不兼容（除非有 <code>const</code> 临时量等特殊情况）

19. 下列关于引用（Reference）的说法，错误的是？

- A. 引用必须在定义时初始化，且一旦绑定到一个对象后，不能再重新绑定到另一个对象。
- B. 对引用取地址（&ref）得到的是它所引用的那个对象的地址，而非引用自身的地址。
- C. 可以定义指向引用的指针（如 `int &* p = &ref;`），这是合法的语法。
- D. 不存在“引用的数组”（如 `int &arr[10];` 是非法的），但可以有“数组的引用”。

正确答案：C

各选项分析：

- 选项 A：正确。引用的本质是“对象的别名”，必须在定义时绑定到对象，且绑定后无法重新绑定到其他对象。
- 选项 B：正确。引用没有独立的内存空间，对引用取地址（&ref）等价于对其绑定的对象取地址，不会得到“引用自身的地址”。
- 选项 C：错误。引用不是独立的“对象”，不具备可被指针指向的实体地址，因此**无法定义“指向引用的指针”**（`int &*p` 是非法语法）。C++ 中指针只能指向“对象”，而引用本身不是对象。
- 选项 D：正确。“引用的数组”（如 `int &arr[10]`）非法（数组元素需为对象，引用不是对象）；但“数组的引用”合法（如 `int arr[10]; int (&ref)[10] = arr;`）。

若 `int a = 10; int &b = a;`，则下列说法正确的是（ ）

C

- A. b 是 a 的副本，修改 b 不影响 a
- B. b 和 a 占用不同的内存空间
- C. 引用 b 必须在声明时初始化
- D. 可以对 b 再次赋值为其他变量的引用

引用和指针的区别

- 指针是一个变量，可以在程序中改变指针所指向的目标；
- 引用是一个声明，引用不是值，不占内存空间，在声明引用时必须给它初始化，并且引用在初始化后，不可以再关联其他的目标对象。
- 引用更安全，因为必须被初始化，指针可能会有野指针

方式	是否拷贝	能否修改原变量	能否传 null	调用语法	适用场景
值传递	✅ 是	❌ 否	❌	<code>f(x)</code>	小类型、不需要修改
指针传递	❌ 否	✅ 是	✅	<code>f(&x)</code>	可选参数、C 接口
引用传递	❌ 否	✅ 是	❌	<code>f(x)</code>	必须修改、高效传大对象
const 引用	❌ 否	❌ 否	❌	<code>f(x)</code>	只读大对象（首选）

下列关于引用和指针的区别，错误的是（ ）

D

- A. 引用必须初始化，指针可以不初始化
- B. 引用一旦绑定变量，不能再绑定其他变量；指针可以指向其他变量
- C. 引用没有自己的地址，指针有自己的地址
- D. 引用可以为 NULL，指针不能为 NULL

10. 在 C++ 中，引用（Reference）与指针的主要区别之一是？

- A. 引用占用内存空间，指针不占用
- B. 引用在初始化后可以重新绑定到另一个对象
- C. 引用本质上是一个自动解引用的常量指针
- D. 指针更安全，引用容易出错

正确答案：C

各选项分析：

- **选项 A：**错误。指针是存储地址的变量，会占用内存空间（如 64 位系统占 8 字节）；引用本质是对象的别名，通常不占用额外内存空间。
- **选项 B：**错误。引用在初始化时必须绑定到一个对象，且绑定后**无法重新绑定到其他对象**，这是引用的核心特性之一。
- **选项 C：**正确。引用的本质可等价于“自动解引用的常量指针”：
 - 语法上，`T &ref = obj;` 等价于 `T *const ptr = &obj;`（`const` 表示指针指向不可修改）；
 - 使用时，引用会自动解引用（无需像指针那样写 `*ptr`），因此是“自动解引用的常量指针”。
- **选项 D：**错误。引用必须初始化（避免空引用），而指针可能出现空指针、野指针等风险，因此**引用比指针更安全**。

下列说法中错误的是？

- A. 表达式 `sizeof(arr)` 返回的是整个数组的字节数（通常是 40），而 `sizeof(ptr)` 返回的是指针本身的大小（通常是 8 或 4）。
- B. 数组名 `arr` 是一个左值（lvalue），但不能被赋值；指针 `ptr` 是一个可以被修改的左值。
- C. 在函数参数列表中，`void func(int a[10])` 和 `void func(int *a)` 声明的是完全相同的函数，因为数组会退化为指针。
- D. 表达式 `&arr` 和 `&arr[0]` 的值（地址）虽然相同，但它们的类型不同：`&arr` 的类型是 `int (*)[10]`，而 `&arr[0]` 的类型是 `int*`。

函数中参数的传递

- 值传递
 - 采用值传递方式时，被调用函数得到的是参数值的副本。
 - 传副本，原变量不受影响（形参不影响实参）
- 指针传递

- 传地址，可通过*p修改原变量，允许为 nullptr，调用时需显式取地址
 - 形参影响实参
 - 引用传递
 - 被调用函数可以直接修改调用函数的原始数值。
 - 别名，可直接修改原变量，不能为空
 - 必须绑定到有效对象，无拷贝开销；安全性高于指针（无空引用概念）。
 - 形参影响实参
6. 函数调用时，实参与形参之间的数据传递方式是 值传递 传递。（注：此处指默认的按值传递）

函数参数传递方式为指针传递时，函数内部对指针的修改会改变实参的值。

错误

指针传递的核心是“函数内部修改指针指向的内容，才会改变实参的值”；若仅修改指针变量本身（如 `p = new int`），不会影响实参指针。例如：`void func(int* p) { *p = 10; }`（改内容，实参变） vs `void func(int* p) { p = nullptr; }`（改指针，实参不变），题干未区分“改指针”和“改内容”，故说法错误。

面向对象编程 OOP

- 以对象为核心组织代码的编程范式
- 用程序模拟现实世界，软件更直观、灵活、可维护

对象

- 面向对象编程（OOP）中最基本、最核心的运行时实体
- 代表现实世界里某个具有状态：数据（变量）和行为：方法（函数）的事物



对象 = 数据（属性） + 操作数据的方法（行为）的封装体

类的作用

①抽象现实世界：

- 类作为对象的抽象，具有该类对象的共同属性和行为。
- 类是多个对象的抽象，对象是类的个体实例。

②封装数据与行为：

- 类把相关的数据（成员变量）和操作（成员函数）捆绑在一起，形成一个逻辑单元。
- 通过访问控制（`private/protected/public`），隐藏内部实现，只暴露安全接口。
- 这使得代码更安全、更易维护：外部不能随意修改对象内部状态

③支持代码复用与扩展：

- 一个类可被多次实例化（创建多个对象）。
- 更重要的是，它可通过**继承**被其他类复用和扩展。

OOP四大特性

OOP 特性	类在其中扮演的角色	C++ 实现机制
抽象 (Abstraction)	类本身就是一种抽象	将对象抽象为类class
封装 (Encapsulation)	通过访问控制包括内部成员：① 数据成员通常设为 private；② 行为通过 public 方法暴露	private / public / protected 访问修饰符
继承 (Inheritance)	通过继承，快速传递已有的功能： ① 基类（父类）定义通用功能； ② 派生类（子类）扩展或重用	class Child : public Parent
多态 (Polymorphism)	多态允许各子类的个性化： ① 基类声明虚函数；② 派生类重写实现；③ 通过基类指针/引用调用不同版本	virtual 函数 + 动态绑定

抽象、封装、继承 面向对象编程的三大特征。

错误	面向对象编程的三大核心特征是“封装、继承、多态”；“抽象”是面向对象的设计思想（类的创建基于抽象），并非与封装、继承并列的三大特征，题干遗漏“多态”、错误加入“抽象”，故说法错误。
----	--

类的声明

- 类是一组变量及其相关函数的组合。
- 类中的变量称为类的**成员变量**或**数据成员**；函数称为类的**函数成员**或方法。

```
class 类名 //class指明这里声明的是一个类类型；类名是新建类的名称，它是一个标识符
{
    public: //公有成员：用关键字 public声明，其中的成员一般是成员函数，用于定义类的外部接口，在程序中的任何部分都可以访问。
        若干成员;
    protected: //保护成员：用关键字protected声明的类成员。
        若干成员;
    private: //私有成员：用关键字 private声明，其中成员一般是 数据成员，用于描述类的属性，它们只能被类自身的成员函数访问。类的成员在默认情况下都是私有的。
        若干成员;
};
```

类成员按照访问权限控制分类

①public**公有成员**：用关键字 public声明，其中的成员一般是成员函数，用于定义类的外部接口，在程序中的任何部分都可以访问（包括类内部、类外部的函数、对象、派生类等），可以直接被对象调用

(obj.public_function());)。

②private**私有成员**：用关键字 private 声明，其中成员一般是 数据成员，用于描述类的属性，它们只能被类自身的成员函数以及友元访问。



类的成员在默认情况下都是私有的。

如果一个成员函数只供类中的其他成员函数调用，也应该声明为私有的。

③protected**保护成员**：用关键字protected声明的类成员。只能被类内部+派生类（子类）访问，类外部包括main函数也不可以



派生类（子类）：**是通过继承机制**，从一个或多个已有的类（称为**基类 / 父类**）创建出的新类。它的核心作用是复用基类的代码，同时可以扩展新功能或修改基类的行为（实现多态（面向对象的三大特性：封装、继承、多态）：同一接口，多种实现（借助虚函数）：基类动物都有“叫”的通用行为，只需要调用“叫”这个通用接口，程序会自动根据到底是哪个派生类（猫？狗？），执行不同的“叫”），是实现“代码复用”“功能扩展”的核心方式之一。

class 派生类名：继承方式 基类名

```
#include <iostream>
using namespace std;

class Animal {
public:
    // 虚函数：运行时确定调用版本
    virtual void make_sound() { cout << "动物叫" << endl; }
    // 析构函数建议加virtual（后续讲）
    virtual ~Animal() {}
};

class Cat : public Animal {
public:
    void make_sound() override { cout << "喵喵喵" << endl; } //重写虚函数
};

int main() {
    Animal* ptr = new Cat();
    ptr->make_sound(); // 运行时绑定：调用Cat的make_sound → 输出“喵喵喵”
    delete ptr;
    return 0;
}
```



在类的声明中，关键字public、protected和private的出现顺序是任意的，可以不出现或多次出现。类中的每个成员只能有一种特定的访问属性。

• 静态成员变量

是 C++ 类中用 `static` 关键字修饰的成员变量，核心特征是：它属于「类本身」，而非类的某个具体对象——所有该类的对象共享同一份静态成员变量，其内存分配在「静态存储区」（而非栈 / 堆），生命周期从程序启动时分配，直到程序结束时释放。

```
//类内声明，类外初始化
class MyClass {
public:
    // 1. 类内声明静态成员变量（static关键字）
    static int public_static_var; // 公有静态成员变量
private:
    static double private_static_var; // 私有静态成员变量（受访问权限约束）
};

// 2. 类外初始化：语法 = 类名::静态成员名 = 初始值;
int MyClass::public_static_var = 10; // 初始化int型静态成员
double MyClass::private_static_var = 3.14; // 即使是private，也必须类外初始化
//初始化时不能加static关键字（类内已声明，类外仅初始化）
```

特性	静态成员变量	非静态成员变量
归属	类级别（所有对象共享）	对象级别（每个对象独有）
内存分配	静态存储区（程序启动时分配）	栈 / 堆（对象创建时分配）
生命周期	程序启动 → 程序结束	对象创建 → 对象销毁
初始化	类内声明，类外初始化	构造函数 / 初始化列表初始化
访问方式	类名:: 成员名（无需对象）	必须通过对象 / 指针访问
共享性	所有对象共享同一份副本	每个对象有独立副本

• 成员函数声明为const

该函数不会修改对象的非 `mutable` 成员变量（仅具备“只读”权限）

`const` 成员函数的本质是限制函数对对象状态的修改，提升代码的安全性与可读性。

关于 C++ 类的访问控制（public/private/protected），下列说法正确的是？

- A. 类的private成员仅能被类的成员函数和友元访问，对象无法直接访问
- B. 类的public成员在类外可通过“类名::成员名”直接访问，无需创建对象
- C. protected成员与private成员完全等价，仅在继承场景下才有区别
- D. 类的成员函数默认是private权限，成员变量默认是public权限

A	<code>private</code>成员的访问规则：仅类的成员函数和友元可访问，类外对象无法直接访问，A 选项正确。 B 选项“<code>public</code> 成员需通过对象访问”（非<code>static</code>成员），“类名：<code>:</code> 成员名”仅用于<code>static</code>成员； C 选项“<code>protected</code> 与 <code>private</code> 非完全等价”（子类可访问父类 <code>protected</code> 成员）； D 选项“类成员默认权限为 <code>private</code>”（无论函数 / 变量），均为错误说法。
	子类继承父类后，无论采用何种继承方式（<code>public/protected/private</code>），都无法直接访问父类的<code>private</code>成员。
正确	父类的<code>private</code>成员是“私有实现细节”，仅允许父类自身的成员函数和友元访问；子类无论采用 <code>public</code>、<code>protected</code> 还是 <code>private</code> 继承，都无法直接访问父类的<code>private</code>成员（需通过父类的 <code>public/protected</code> 成员函数间接访问），符合 C++ 访问控制规则，故说法正确。

类的定义（实现）

对于成员函数，类的声明内一般只声明其函数原型，具体函数的实现要在类外定义。

```
返回值类型 类名::成员函数名（参数表）    // "::"叫作用域运算符，它标明所要定义的函数属于哪个类。
{
    函数体
}
```

类的存储

在实际的C++程序设计中，把类的声明和类外实现的成员函数放在不同的文件中。其中类的声明放在一个称为头文件的文件中，成员函数的定义放在另一个称为源文件的文件中。

头文件与源文件的名字一般相同。

头文件的扩展名为.h；源文件的扩展名为.cpp。

对象的创建

创建对象就是根据一个类（Class）生成一个具体的实例（Instance）。这个过程也叫“实例化”。创建对象的过程总一定会调用构造函数。

类名 对象名；



类是对象的抽象和描述，对象是类的实例

```
Cylinder cylinder1,cylinder2; //创建了两个名字分别为cylinder1、cylinder2的类Cylinder的对象。
Cylinder cylinders [10] ; //创建了一个数组cylinders，其中共有10个元素，每个元素都是一个对象。
```

对象如何获得类的成员？

对象通过“点运算符（`.`）”或“箭头运算符（`->`）”来访问类中定义的数据成员和成员函数。`this`来访问当前对象的指针。但能否访问，取决于这些成员的访问权限（`public / private / protected`）。

```

class Dog {
private:
    int age;
public:
    string name;
    void bark() { cout << name << " says Woof!\n"; }
};

Dog myDog;
// 访问公有数据成员
myDog.name = "Max";
// 调用公有成员函数
myDog.bark();

Dog* pDog = new Dog("Lucky");
// 等价于 (*pDog).name
pDog->name = "Sparky";
// 等价于 (*pDog).bark()
pDog->bark();
delete pDog;

Dog myDog;
// 访问私有数据成员
myDog.age = 5; // ❌❌❌ 私有数据成员无法直接访问，需要通过公有函数
// 访问公有数据成员
myDog.name = "Max";
// 调用公有成员函数
myDog.bark()

```



- 对象如何访问成员？
用 .（对象）或 →（指针）
- 对象能访问哪些成员？
仅 public 成员（数据或函数）
- 对象如何访问和操作私有数据？
通过公有成员函数间接访问和操作
- 成员函数内部能访问哪些成员？
所有成员
- 成员函数内部怎么访问数据？
直接使用

对象是如何产生的？它有生命吗？

一、对象的产生过程

- ①分配内存 `Cylinder c;`
- ②调用构造函数（出生证明）：初始化



构造函数是在创建对象时自动调用的特殊成员函数，用于初始化对象的数据成员。它的名字必须与类名相同，并且没有返回类型（连 `void` 都不能有）。

• 构造函数

- 构造函数应被声明为公有函数（但也可以声明为`private`），它只能在创建类的对象时由系统自动调用，对对象进行初始化。
- 构造函数不声明返回类型

如果没有显式定义任何构造函数，编译器会自动生成一个默认无参构造函数（当类显式定义了带参构造函数时，编译器不会自动生成默认无参构造函数），该构造函数不带参数，函数体为空，用于创建对象（C++编译器在类未定义对应函数时，会隐式生成默认函数（包括：无参默认构造函数，拷贝构造函数，拷贝赋值运算符，析构函数））

```
//不带参数（默认）的构造函数
Cylinder( ) {
    radius=1;
    height=1;
}

//带参数的构造函数
Cylinder(double r, double h) {
    radius=r;
    height=h;
}

//拷贝构造函数：类名(const 类名&);
Cylinder(const Cylinder& other) {
    radius=other.radius;
    Height=other.height;
}

//拷贝赋值运算符（不属于构造函数）
```

构造函数**支持重载**，一个类中可以定义多个构造函数，它们的参数表（个数、类型、顺序）不同，用于不同方式初始化对象

```
Cylinder(){}
Cylinder(string name){}
Cylinder(string name, int name){}
```

二、对象的销毁过程

(自动)调用析构函数 `~Cylinder()` 清理资源；
释放栈内存。

- **析构函数**



析构函数是在对象生命周期结束时自动调用的特殊成员函数，主要用于释放资源（如关闭文件、释放动态内存等）。
名字前加波浪号（~），且不能有参数。

```
~ Cylinder() {  
    cout << "MyClass object destroyed!" << endl;  
}
```

如何调用？

- 离开作用域时自动销毁；
- `new` 创建的对象/对象数组，通过 `delete` 手动销毁

- **拷贝赋值运算符**

它是类的特殊成员函数之一，核心作用是**将一个已存在对象的值，赋值给另一个同类型的已存在对象**（注意：是“赋值”，而非“创建新对象”）

```
类名& operator=(const 类名& 源对象) {  
    // 赋值逻辑（将源对象的成员赋值给当前对象）  
    return *this;  
}
```

如果类**未显式定义拷贝赋值运算符**，编译器会自动生成一个**默认拷贝赋值运算符**，并进行逐成员浅拷贝

创建对象的方式

1. 直接声明栈对象

不通过动态分配内存的方式，直接在函数/代码块作用域内声明的对象

2. `new`动态创建堆对象

3. 静态存储区对象（`static`关键字）

- 静态存储区的变量（`static` 修饰）在程序启动时分配内存，程序结束时释放
- `static`成员变量属于类本身，而非类的某个对象，需在类外单独初始化
- **✗**普通全局变量加`static`后，存储位置从堆区变为静态存储区

✓普通全局变量本身就存于静态存储区，static仅修改其“链接属性”(从外部链接变为内部链接，限制作用域为当前源文件)，不改变存储位置

- static修饰的函数无法被其他源文件通过extern关键字引用

4. 全局对象（声明在所有函数外）

直接创建的栈对象，其生命周期有作用域控制，进入作用域时构造，离开作用域时自动析构，而对下个占用的栈内存会被系统自动释放

已经是成员的不需要再写在参数表里了

- 关于构造函数，以下说法错误的是:
- A. 构造函数没有返回类型
- B. 构造函数可以重载
- C. 构造函数可以是 private 的
- D. 构造函数必须显式定义，否则无法创建对象

D

析构函数的作用是:

- A. 初始化对象的数据成员
- B. 释放对象占用的资源
- C. 复制另一个对象的内容
- D. 返回对象的大小

B

以下哪个符号用于通过对象指针访问成员函数？

- A. .
- B. ::
- C. ->
- D. *

C

成员函数声明为 const 表示:

- A. 该函数返回常量
- B. 该函数不能修改对象的任何成员
- C. 该函数只能被 const 对象调用
- D. 该函数不能有参数

B

- 假设 MyClass 有一个带参构造函数 MyClass(int x)，但没有默认构造函数。以下哪行代码会编译失败？
- A. MyClass a(5);
- B. MyClass b = MyClass(10);
- C. MyClass c;
- D. MyClass* p = new MyClass(3);

C

使用 new 创建的对象，其析构函数在何时被调用？

- A. 程序结束时自动调用
- B. 离开作用域时自动调用
- C. 调用 delete 时
- D. 对象赋值给另一个对象时

C

继承与派生

- 面对对象编程的四大基本特征：
抽象、封装、继承、多态

继承和派生

- 继承和派生是同一个概念的两种视角描述，它们是面对对象编程的核心机制，允许一个类获取另一个类的属性和方法。
- 继承：从父类的角度看，子类继承了父类的特性
- 派生：从子类的角度看，子类是从父类派生出来的

继承：建立类之间的层次关系，使得某类可以继承另外一类的特征和能力，在已有的类的基础上建立一个新的类

已有的类：基类父类

新的类：派生类子类

一个派生类可以从一个父类也可以从多个父类派生，多继承

派生类的声明与实现

从一个父类派生一个子类的一般格式：

class 派生类名：<继承方式> 基类名

```
class ClassName:<Access>BaseClassName
{
    //派生类中新增加的成员
    private:
    ...;    //私有成员说明
    public:
    ...;    //公有成员说明
```

```
protected:
    ...;    //保护成员说明
};
```

- 在建立派生类的过程中，父类不会做任何改变
- 派生类除了继承父类的所有可引用的成员变量和成员函数外，还可以另外定义本身的成员变量和处理这些变量的函数
- 由于派生类可继承父类的成员变量和成员函数，因此在父类中定义好的数据和函数等的程序代码可重复使用

类的继承方式

继承方式控制着基类成员在派生类中的访问权限（vs访问属性）

- 公有继承
- 保护继承
- 私有继承

不同继承方式下的成员访问属性

- 公有继承
 - 基类的私有成员不能被派生类的函数成员访问
 - 基类的公有成员和保护成员在派生类中的访问权限不变
- 保护继承
 - 基类的私有成员不能被派生类的函数成员访问
 - 基类的公有成员和保护成员在派生类中的访问权限变为保护类型
- 私有继承
 - 基类的私有成员不能被派生类的函数成员访问
 - 基类的公有成员和保护成员在派生类中的访问权限变为私有类型
 - 基类：public: (变为私有)在派生类中使用，类外不可使用
 - protected: (变为私有) 在派生类中使用，类外不可使用
 - private: 不能在派生类中和类外使用

派生类的默认访问属性

默认是私有的

构造和析构的顺序

- 构造函数调用顺序
 1. 基类构造函数（按声明顺序）
 2. 成员对象构造函数（按声明顺序）
 3. 派生类构造函数体（按声明顺序）

- 构造函数执行顺序（与构造完全相反）：

1. 派生类析构函数体
2. 成员对象析构函数（按声明逆序）
3. 直接基类析构函数（按声明逆序）
4. 虚拟基类析构函数（按声明逆序）

使用class关键字定义派生类时，默认继承方式是：

- A. public
- B. protected
- C. private
- D. 不确定

C

私有继承时，基类的protected成员在派生类中变为：

- A. public
- B. protected
- C. private
- D. 不可访问

C

保护继承时，基类的public成员在派生类外部：

- A. 可以访问
- B. 不能访问
- C. 有时可以访问
- D. 通过派生类对象可以访问

B

多态

- 多态的概念
 - 同一接口，不同实现（一个接口，多种方法）
 - 允许通过基类指针/引用调用派生类函数
 - 运行时根据实际对象类型确定调用哪个函数



多态的实现

- 虚函数是 C++ 中实现运行时多态（动态绑定）的核心机制。它允许在基类中声明一个函数为“虚”的，使得派生类可以重写（override）该函数，并在通过基类指针或引用调用该函数时，实际执行的是派生类中的版本。
- 使用关键字 virtual 在类的成员函数声明前将其声明为虚函数

```

class Base {
public:
    virtual void func() {
        cout << "Base::func()" <<
    endl;
    }
}
  
```

- 只能用于类的非静态成员函数
- 构造函数不能是虚函数（但析构函数可以，而且通常强烈建议设为虚函数）

- 一旦在基类中将某函数声明为virtual，那么在所有派生类中该函数自动成为虚函数（不用加virtual关键字）
- 虚函数可以有默认实现，也可以没有实现，则称为纯虚函数。

多态的实现

```
#include <iostream>
using namespace std;

class Animal{
public:
    virtual void speak(){
        cout<<"Animal speaks"<<endl;
    }
};

class Dog:public Animal{
public:
    void speak() override{
        cout<<"Dog barks"<<endl;
    }
};

int main(){
    Animal*a=new Dog();
    a->speak();
    delete a;
    return 0;
}
```

- 虚函数表机制
 - C++编译器为每个包含虚函数的类生成一个虚函数表，其中存储了该类所有虚函数的地址。每个对象内部包含一个隐藏指针，指向所属类的vtable。
 - 当通过基类指针调用虚函数时，程序会：
 - 通过对象的 vptr 找到其 vtable；
 - 在 vtable 中查找对应函数的地址；
 - 调用该地址处的函数（即派生类的实现）



注意：这种机制带来了一定的性能开销（间接调用、额外内存），但这是实现多态所必需的。

纯虚函数

```
virtual void f()=0; //用于定义接口
```

如果希望基类不提供具体实现，强制派生类必须实现该函数，可以定义纯虚函数

- 含有纯虚函数的类称为**抽象类**，不能实例化
用于定义抽象类的机制是包含**纯虚函数**
- 派生类必须实现所有纯虚函数，否则它也是抽象类。

虚析构函数

当通过基类指针删除派生类对象时，必须将基类的析构函数声明为虚函数，否则只会调用基类析构函数，导致派生类部分未被正确析构（资源泄露）。

```
#include <iostream>
using namespace std;
class Base {
public:
    virtual ~Base() { cout << "Base destroyed" << endl; }
};
class Derived : public Base {
public:
    ~Derived() { cout << "Derived destroyed" << endl; }
};
int main() {
    Base* p = new Derived();
    delete p; // 正确：先调用 Derived::~~Derived(), 再调用 Base::~~Base()
}
```



若 ~Base() 不是虚函数，只输出 "Base destroyed"，Derived 的析构函数不会被调用！

说明方式	在成员函数前加 virtual
绑定时机	运行时（动态绑定）
实现机制	虚函数表（vtable）+ 虚指针（vptr）
多态前提	通过基类指针/引用调用虚函数
析构函数	建议基类析构函数设为虚函数
纯虚函数	virtual void f() = 0;，用于定义接口

- 构造函数不能是虚函数：因为构造对象时需要知道确切类型。
- 静态成员函数不能是虚函数：因为静态成员函数属于类，而不属于对象。
- 虚函数可以是内联函数：但一旦被调用通过多态机制，内联展开就会失效
- 虚函数的调用成本：通过指针或引用调用虚函数比直接调用非虚函数多一次间接寻址（查虚函数表）。

默认参数父类和子类有冲突的话，是按照父类

是什么时候使用虚函数？

- ①需要“通过基类接口操作派生类对象”时
- ②设计“一个可扩展的框架或库”时
- ③希望提供“默认行为”。但允许子类覆盖时

在C++中，静态联编和动态联编的主要区别是什么？

- A. 静态联编是在编译时确定函数调用的具体实现，而动态联编是在运行时确定。
- B. 静态联编是通过指针或引用来调用函数，而动态联编是通过虚函数表来调用。
- C. 静态联编允许重载，而动态联编允许多态。
- D. 静态联编是在编译时进行类型检查，而动态联编是在运行时进行类型检查。

A 静态联编是编译期确定函数调用（如函数重载、普通函数调用）；动态联编是运行期确定函数调用（如虚函数 + 父类指针 / 引用），A 选项正确。
 B 选项“动态联编通过虚函数表”（是实现方式，非核心区别）；
 C 选项“静态联编允许重载”（是表现，非区别）；
 D 选项“类型检查时机”（非联编的核心区别），均为错误说法。

C++ 动态多态（虚函数实现）的关键是 “编译期确定调用的函数”，而静态多态（函数重载）是 “运行期确定”。子类重写父类虚函数时，必须显式添加virtual关键字，否则无法触发多态。

错误 二者逻辑完全颠倒：①动态多态（虚函数 + 父类指针 / 引用）是运行期根据对象实际类型确定调用的函数；②静态多态（函数重载、模板）是编译期根据参数列表确定调用的函数。且子类重写父类虚函数时，无需显式添加virtual（会自动继承虚属性），仅需保证函数签名一致，故说法错误。

Vector（概念了解）

- std::vector 是 C++ 标准模板库（STL）中最常用、最重要的动态数组容器之一，定义在头文件 <vector> 中。它提供了连续内存存储、自动内存管理和丰富的操作接口，是替代原生数组的首选。
- std::vector
 是 C++ 中最高效、最安全的动态数组实现。只要你需要“可变长度 + 快速随机访问”的序列容器，vector 几乎总是第一选择。

特性	说明
动态大小	可在运行时自动扩容（无需手动 new/delete）
连续存储	元素在内存中连续存放，支持随机访问（O(1)）
模板类	支持任意类型：vector<int>, vector<string>, vector<MyClass> 等
自动管理内存	析构时自动释放资源（RAII 原则）
支持迭代器	可与 STL 算法（如 sort, find）无缝配合

- 初始化

```
#include <vector>
vector<int> v1;           // 空
vector<int> v2(5);         // 5 个元素，初始化为0
vector<int> v3(5, 10);     // 5 个元素，每个都是10
vector<int> v4{1, 2, 3, 4, 5}; // 列表初始化（C++11）
vector<int> v5 = {1, 2, 3}; // 同上
```

操作	时间复杂度
随机访问	v[i]
尾部插入/删除	push_back / pop_back
中间插入/删除	insert / erase
扩容机制	—

是否需要虚函数？

- 希望提供“默认行为”，但允许子类覆盖时

- ①基类定义接口，派生类实现不同行为 **是**
- ②基类提供默认行为，允许覆盖 **是**
- ③基类仅作为借口（无实现）**是**
- ④类会被继承，且可能通过基类指针删除对象 **是**，虚函数得是虚函数
- ⑤所有子类行为一样 **否**

析构函数要多态，不然会资源释放不完整

虚函数=0没有内容，但是对于析构函数来说不能=0，父类也要释放，所以会用=default

特殊情况下，纯虚函数可以有函数体，有公共东西需要执行，但子类必须重写

以下关于 C++ 虚函数的说法中，哪些是正确的？（可多选）

- A. 构造函数可以是虚函数
- B. 析构函数可以是虚函数 ✓
- C. 虚函数必须在派生类中重写
- D. 使用 override 关键字可以防止意外未重写基类虚函数 ✓
- E. 虚函数调用一定比普通函数慢 ✓

- 下列代码是否合法？

```
class A { public: virtual ~A() = default; };
class B : public A { public: ~B() override {} };
```
- A. 合法
- B. 不合法，析构函数不能用 override
- C. 不合法，~A() 不能是虚函数
- D. 不合法，= default 不能用于虚函数

• 答案：A

关于纯虚函数，正确的是？

- A. 含纯虚函数的类可以实例化
- B. 纯虚函数不能有函数体
- C. 派生类必须实现所有纯虚函数，否则仍是抽象类
- D. 纯虚函数不能是 protected 的

答案：C

选项 A：错误。含纯虚函数的类是抽象类，抽象类不能实例化（只能作为基类被继承）；

选项 B：错误。纯虚函数可以有函数体（如 `virtual void f() = 0;` 后可在类外定义 `void A::f() {}`），只是不能依赖它实例化类；

选项 C：正确。派生类若未实现基类的所有纯虚函数，自身仍为抽象类，无法实例化；

选项 D：错误。纯虚函数可以是 `protected`（仅限制外部访问，不影响抽象类的定义）。

运算符重载（概念了解）

- 目的：让自定义类型像内置类型（如 `int`、`string`）一样使用运算符。
为自定义类型（类或结构体）重新定义（重载）已有运算符（如 `+`、`==`、`<<` 等）的行为，使代码更直观、自然。
- 本质：运算符重载就是函数重载，只是名字特殊（如 `operator+`）。
- 重载方式：
 - 采用成员函数方式进行运算符重载
 - 左操作数是当前对象（`*this`），右操作数是参数
 - `MyClass operator+(const MyClass& other) const;`
 - 采用非成员函数方式进行运算符重载
 - 左操作数不是当前对象（`*this`），就必须用非成员函数。
 - `friend std::ostream& operator<<(std::ostream& os, const Person& p);`

友元（概念了解）

- 友元（friend）是 C++ 提供的一种机制，允许一个函数或类访问另一个类的 `private` 和 `protected` 成员，即使它不是该类的成员函数或派生类。
- 友元函数（非成员函数）
- 友元类
- 友元成员函数（另一个类的特定成员函数）

特性	说明
单向性	A 是 B 的友元 $\neq$ B 是 A 的友元
不具有传递性	A 是 B 的友元，B 是 C 的友元 $\Rightarrow$ A 不是 C 的友元
不具有继承性	派生类不能继承基类的友元关系
破坏封装	滥用会降低代码安全性与可维护性
不是成员函数	友元函数没有 <code>this</code> ，调用方式如普通函数

文本文件读写

- 核心思想是——把文件当作“流”来处理，就像 `cin` 和 `cout` 一样自然。
- 主要通过标准库中的 `<fstream>` 头文件提供的三个类实现：
 - `std::ifstream`：用于读取文本文件（input file stream）

- `std::ofstream`: 用于写入文本文件 (output file stream)
- `std::fstream`: 可同时用于读写 (file stream)
- 读取文件

方法	说明
<code>.is_open()</code>	文件是否成功打开
<code>.good()</code>	流状态正常 (无错误)
<code>.fail()</code>	读写失败 (如格式错误)
<code>.eof()</code>	是否到达文件末尾 (慎用!)
<code>.bad()</code>	发生严重错误 (如磁盘故障)

```
#include <fstream>
#include <iostream>
#include <string>
using namespace std;

int main() {
    ifstream inFile("data.txt");
    ifstream inFile("C:\\Users\\你的用户名\\Desktop\\data.txt");

    if (!inFile.is_open()) {
        cerr << "无法打开文件! \n";
        return 1;
    }

    string line;
    while (getline(inFile, line)) {
        cout << "读取: " << line << endl;
    }

    inFile.close();
    return 0;
}
```

- 写入文件

```
#include <fstream>
#include <iostream>
using namespace std;

int main() {
    // 方式1: 构造时打开
    ofstream outFile("data.txt"); // 默认 ios::out 模式
    // 方式2: 先创建再打开
```

```

// ofstream outFile;
// outFile.open("data.txt");

if (!outFile.is_open()) {
    cerr << "无法创建文件! \n";
    return 1;
}

// 写入数据（使用 << 操作符，像 cout 一样）
outFile << "Hello, world!\n";
outFile << "数字: " << 42 << "\n";
outFile << "浮点数: " << 3.14159 << endl;

// 关闭文件（析构函数会自动关闭，但显式关闭更清晰）
outFile.close();

return 0;
}

ofstream outFile("data.txt", ios::out); //默认，覆盖写入
ofstream outFile("log.txt", ios::app); // 追加模式（不覆盖原有内容）

```

异常处理（概念了解）

- 用于处理运行时错误的机制，它允许程序在发生异常情况（如文件打不开、内存分配失败、除零等）时，安全地转移控制流，避免程序崩溃，并提供清晰的错误恢复路径。
- C++ 异常处理由三个关键字组成
 - Try：可能抛出异常的代码块
 - Throw：抛出一个异常对象
 - Catch：捕获并处理特定类型的异常

```

try {
    // 可能出错的代码
    if (error_condition) {
        throw exception_object; // 抛出异常
    }
}
catch (Type1 e) {
    // 处理 Type1 类型异常
}
catch (Type2 e) {
    // 处理 Type2 类型异常
}
catch (...) {

```

```
// 捕获所有其他类型异常（兜底）
}
```

```
#include <iostream>
#include <fstream>
#include <stdexcept> // 标准异常类头文件

using namespace std;

double divide(int a, int b) {
    if (b == 0) {
        throw invalid_argument("除数不能为零！");
    }
    return static_cast<double>(a) / b;
}

void readFile(const string& filename) {
    ifstream file(filename);
    if (!file.is_open()) {
        throw runtime_error("无法打开文件: " + filename);
    }
    cout << "文件打开成功! \n";
    // ... 读取内容
}

int main() {
    try {
        double result = divide(10, 0); // 会抛出异常
        readFile("nonexistent.txt");
    }
    catch (const invalid_argument& e) {
        cerr << "参数错误: " << e.what() << '\n';
    }
    catch (const runtime_error& e) {
        cerr << "运行时错误: " << e.what() << '\n';
    }
    catch (const exception& e) { // 捕获所有标准异常
        cerr << "标准异常: " << e.what() << '\n';
    }
    catch (...) { // 捕获非标准异常（如 int、char* 等）
        cerr << "未知异常! \n";
    }
    cout << "程序继续运行...\n";
    return 0;
}
```